



CSCI 232

232 Algorithms & Data Structures

LECTURE: 11

J. L. Pach

TODAY'S AGENDA

- ✂ UML Introduction
- ✂ Class Diagram Basics
- ✂ Relationships Overview
- ✂ Multiplicity / Cardinality
- ✂ Encapsulation and Abstraction

UML

Unified Modeling Language

WHAT IS UML?

UML = Unified Modeling Language

- ✚ Universal language for modeling systems
- ✚ Language-independent - works with Java, C#, C++, Python
- ✚ Learn once > apply everywhere
- ✚ Visual representation of classes and relationships

HISTORY OF UML

UML was created in 1997 by three leading software engineers: **Grady Booch**, **James Rumbaugh**, and **Ivar Jacobson**. They unified their three competing object-oriented modeling notations into a single standard.

UML became an **Object Management Group (OMG) standard** in 1997 and has since become the de facto industry standard for software modeling and design documentation.

Current Version: UML 2.5.1 (released in December 2017)

- ✂ The latest stable version widely used in industry
- ✂ UML 2.6 and beyond are in development
- ✂ Backward compatible with previous versions

OBJECT-ORIENTED PROGRAMMING PARADIGMS

✂ UML is built on four core principles of Object-Oriented Programming (OOP):

1. **Encapsulation** (Information Hiding)
2. **Inheritance** (Code Reuse)
3. **Polymorphism** (Many Forms)
4. **Abstraction** (Simplification)

✂ UML Class Diagrams visualize all four principles:

- ✂ **Encapsulation** > Access modifiers ( ,  , )
- ✂ **Inheritance** > IS-A relationships ( / )
- ✂ **Polymorphism** > Virtual methods and overriding
- ✂ **Abstraction** > Abstract classes and interfaces

Encapsulation(Information Hiding)

- ✂ Bundle data (attributes) and methods together in a class
- ✂ Hide internal implementation details
- ✂ Control access through public/private/protected modifiers
- ✂ **Why?** Protects data, prevents unwanted modifications, simplifies interface

Inheritance (Code Reuse)

- ✂ Child classes inherit from parent classes
- ✂ Reuse common functionality
- ✂ Create class hierarchies (IS-A relationships)
- ✂ **Why?** Reduces code duplication, promotes consistency, easier maintenance

Polymorphism (Many Forms)

- ✂ Same interface, different implementations
- ✂ Objects can be used interchangeably
- ✂ Methods behave differently based on object type
- ✂ **Why?** Flexibility, extensibility, cleaner code

Abstraction (Simplification)

- ✂ Hide complexity behind simple interface
- ✂ Focus on "what" not "how"
- ✂ Use abstract classes and interfaces
- ✂ **Why?** Makes code easier to understand, reduces coupling

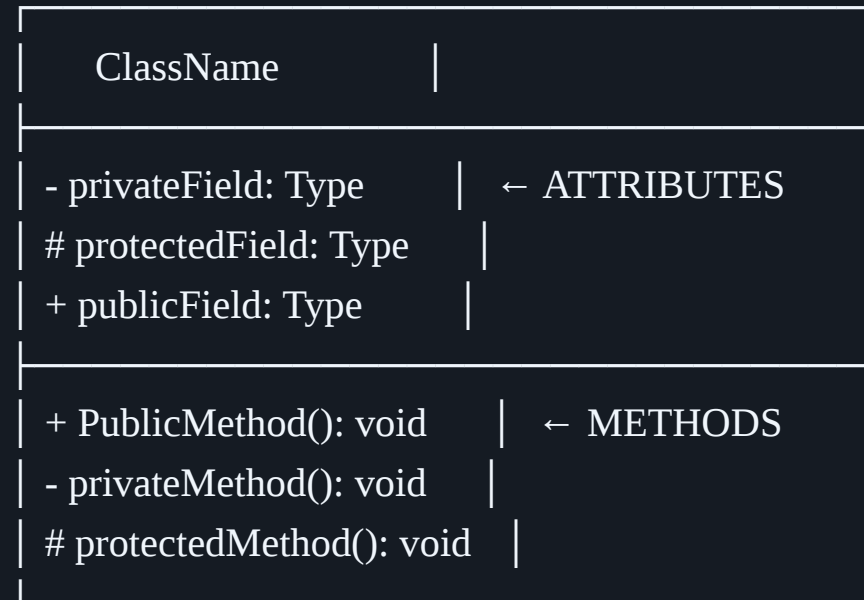
WHY UML BEFORE CODE?

- ✖ See the architecture BEFORE coding
- ✖ Understand relationships
- ✖ Plan encapsulation
- ✖ Catch design problems early
- ✖ Communicate design with team members

UML CLASS DIAGRAM STRUCTURE

Three sections:

1. Class name
2. Attributes (fields/data)
3. Methods (functions)



ACCESS MODIFIERS - NOTATION

Symbol	Access	Visibility
	public	Everyone can see
	private	Only the class
	protected	Class + subclasses

Key Rules:

-   = Public (API)
-   = Private (Hidden)
-   = Protected (For inheritance)

EXAMPLE - BANKACCOUNT CLASS

Convention:

- ✂ Fields are almost always `-` (private)
- ✂ Methods can be `+` (public API*) or `-` (internal helpers)

*application programming interface

```
BankAccount |  
|  
| - balance: double |  
| - accountNumber: string |  
| - owner: string |  
|  
| + deposit(amount): void |  
| + withdraw(amount): void |  
| + getBalance(): double |  
| - validateAmount(): bool |  
|
```

TYPES AND RETURN VALUES

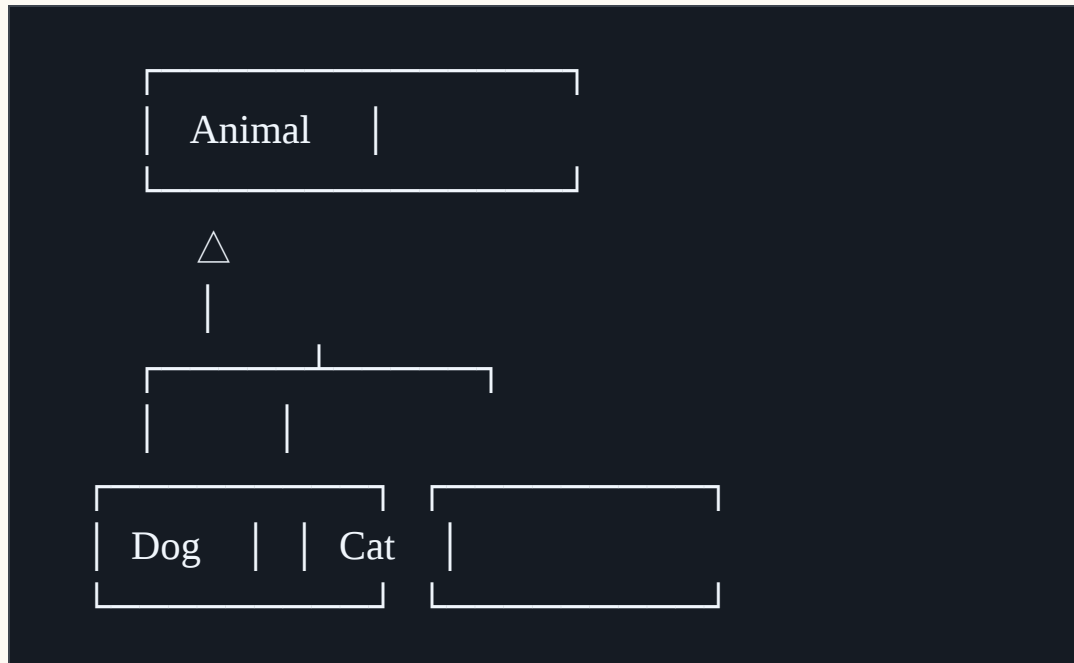
Student

- name: string
- age: int
- grades: vector<double>

+ getName(): string
+ getAverage(): double
+ addGrade(g: double): void
- validate(): bool

Format: methodName(param: Type):
ReturnType

RELATIONSHIPS - IS-A (INHERITANCE)

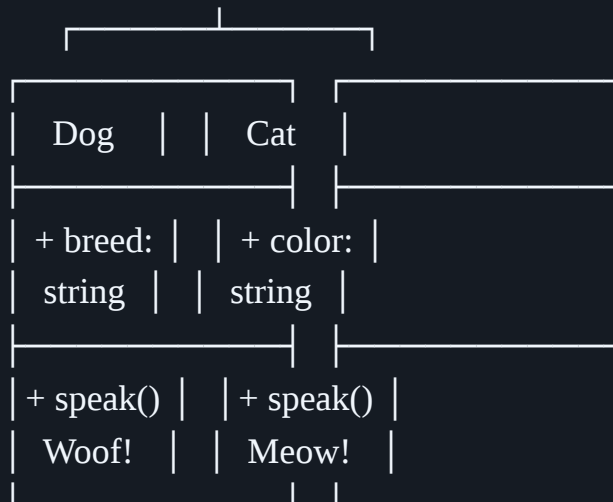
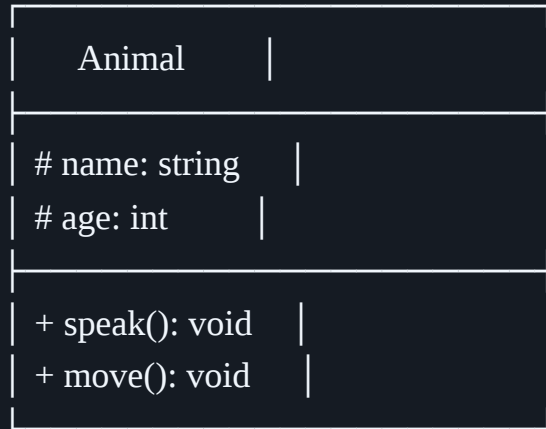


Read: *Dog IS-A Animal*

Arrow points UP to parent class

Empty triangle = public inheritance

Inheritance Example - Detailed



✂ Dog inherits:

✂ name , age , speak() , move()

✂ Dog overrides:

✂ speak() (different implementation)

✂ Dog adds:

✂ breed (specific to dogs)

RELATIONSHIPS BETWEEN OBJECTS

1. IS-A (Inheritance)
2. HAS-A (Ownership/Containment)
 - i. Composition
 - ii. Aggregation
3. USES-A (Collaboration)
 - i. Association
 - ii. Dependency

IS-A & HAS-A

1. IS-A (Inheritance)

- ✂ Parent-child relationship
- ✂ Symbol: $\triangle$ (empty triangle)
- ✂ Example: Dog IS-A Animal

2. HAS-A (Ownership/Containment)

- ✂ One object owns/contains another

COMPOSITION (STRONG HAS-A)

- ✂ Symbol: $\blacklozenge$ (filled diamond)
- ✂ Lifetime dependency
- ✂ Example: Person HAS-A Heart

AGGREGATION (WEAK HAS-A)

- ✂ Symbol: $\diamond$ (empty diamond)
- ✂ Independent objects
- ✂ Example: Team HAS-A Player

USES-A (COLLABORATION)

One object uses/knows about another

ASSOCIATION

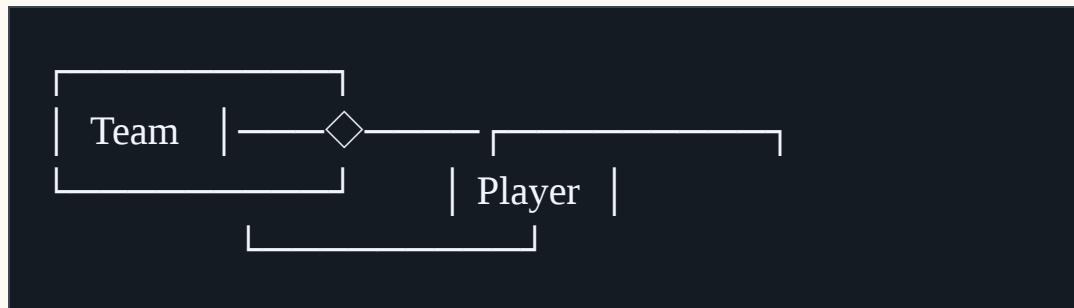
- ✂ Symbol: — (solid line)
- ✂ Bidirectional relationship between independent objects
- ✂ Example: Student enrolls in Course
(`Student has vector<Course*>`)
- ✂ Many-to-many possible

DEPENDENCY

- ✂ Symbol: ---- (dashed line)
- ✂ Temporary usage
- ✂ Weakest coupling
- ✂ Example: Logger uses File (as parameter `void log(File& file)`)

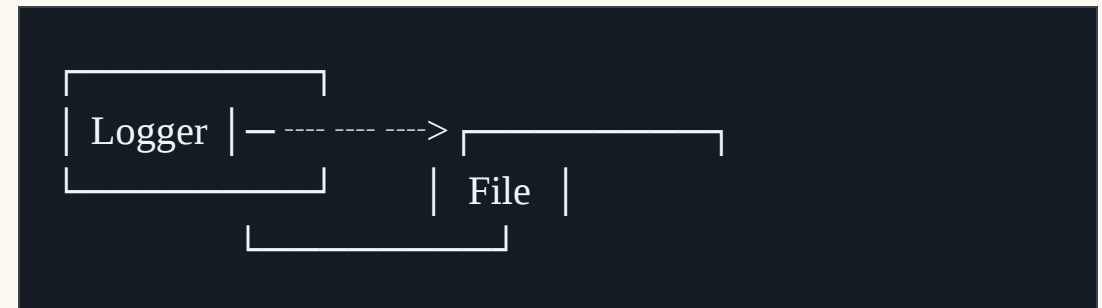
HAS-A VS USES-A

Aggregation (HAS-A - weak)



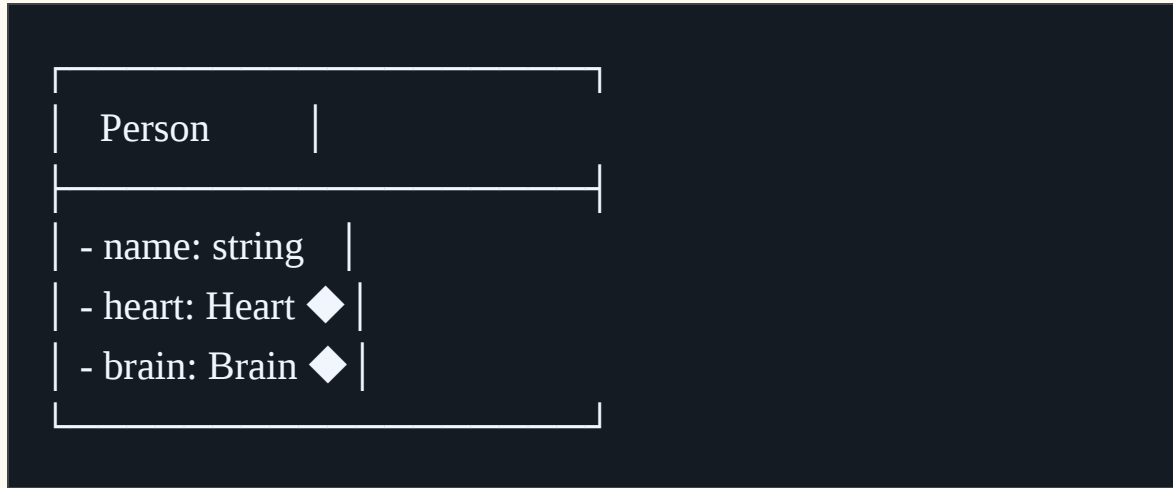
- ✂ Team OWNS Players (but weakly)
- ✂ Field: `vector<Player*> players;`
- ✂ Player can exist without Team

Dependency (USES-A)



- ✂ Logger USES File temporarily
- ✂ Function: `void log(File& f)`
- ✂ File is NOT a field of Logger

Composition (Strong Dependency)



```
class Person
{
    private:
        // Composition - Heart created with Person
        Heart heart;
        Brain brain;
}; // When Person destroyed > Heart
// and Brain destroyed automatically
```

Characteristics:

- ✂ Filled diamond  = Composition
- ✂ Strong ownership: when Person dies > Heart, Brain are destroyed
- ✂ Part cannot exist without whole
- ✂ One-to-one or one-to-many

Read: *Person HAS-A heart*

AGGREGATION (WEAK DEPENDENCY) 1/2



Characteristics:

- ✖ Empty diamond (◇) = Aggregation
- ✖ Weak relationship: **Team** and **Players** are independent
- ✖ When **Team** is deleted > **Players** still exist
- ✖ Many-to-many or one-to-many

Read: *Team HAS-A Player*

AGGREGATION (WEAK DEPENDENCY) 2/2

```
class Team
{
    private:
        vector<Player*> players; // Aggregation - only storing pointers
};
// When Team destroyed > Players still exist somewhere
```



MULTIPLICITY / CARDINALITY

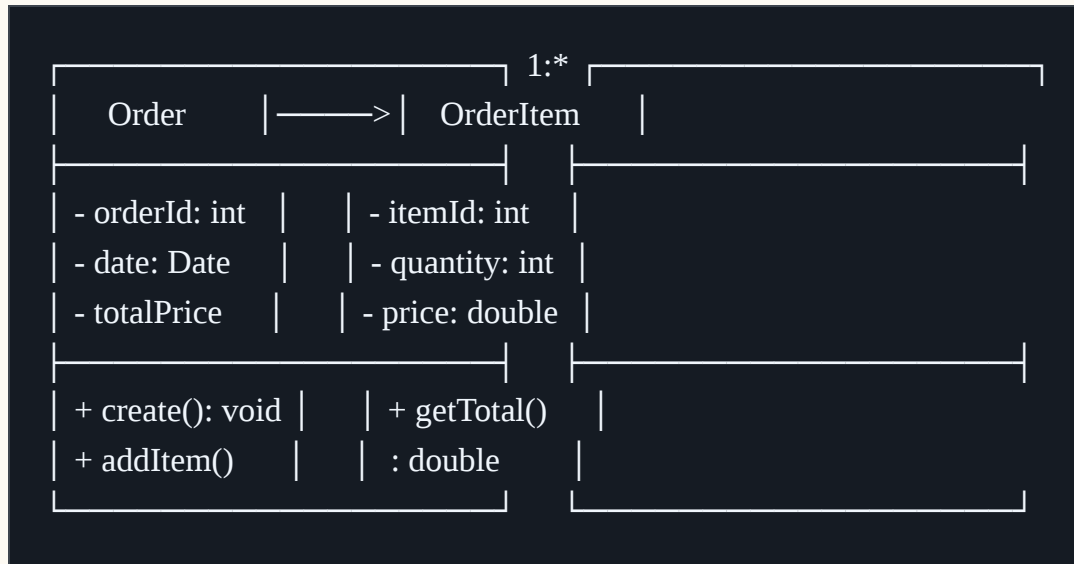
Defines how many objects participate in the relationship

Notation	Name	Meaning	Example
1	One	Exactly one	Person has 1 Passport
0..1	Optional	Zero or one	Person may have 0 or 1 Nickname
*	Many	Zero or more	Order has 0+ OrderItems
0..*	Many (explicit)	Zero or more	Person has 0+ Children
1..*	One or more	At least one	Team has 1+ Players
2..5	Range	Specific range	Car has 2-5 Doors
n	Exactly N	Specific number	Deck has 52 Cards

RELATIONSHIP TYPES - COMPLETE

Type	Symbol	Line	Cardinality	Example	Lifetime	C++ Pattern	When to Use
Inheritance		Solid	1:1	Dog IS-A Animal	N/A	<pre>class Dog : public Animal { }</pre>	IS-A relationship, shared behavior
Composition		Solid	1:1 or 1:*	Person HAS-A Heart	Strong	<pre>Heart heart;</pre>	Part-whole, lifetime dependency
Aggregation		Solid	1:* or *:*	Team HAS-A Player	Weak	<pre>vector<Player*> players;</pre>	Loose coupling, independent objects
Association		Solid	:	Student enrolls Course	Independent	<pre>vector<Course*> courses;</pre>	Relationships between independent objects
Dependency		Dashed	Varies	Logger uses File	Temporary	<pre>void log(File& file)</pre>	Temporary usage, parameters

ORDER AND ORDERITEM



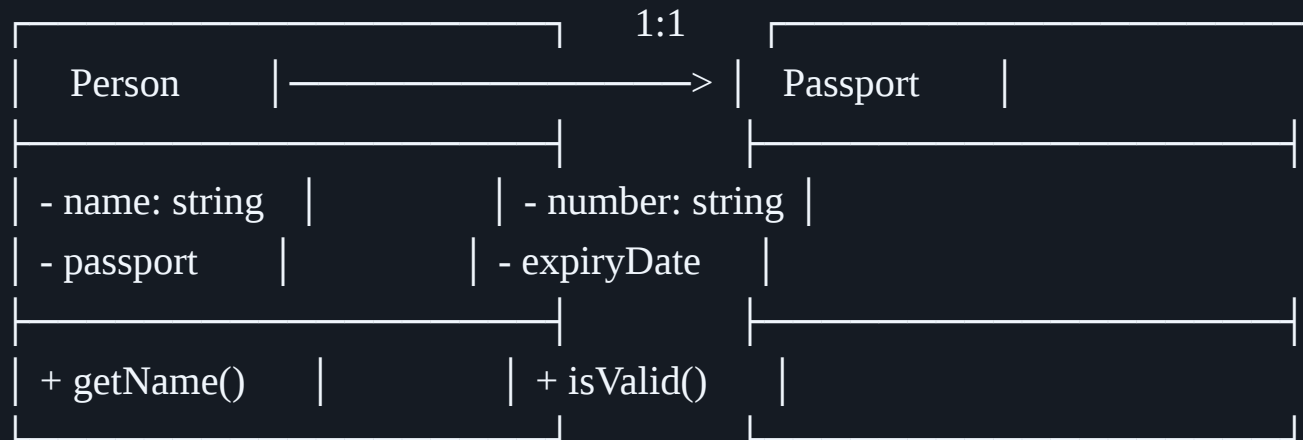
Read:

- ✂ One **Order** HAS-A many **OrderItems** (**1:***)
- ✂ One **OrderItem** belongs to exactly one **Order**

What it means:

- ✂ **Order** can have 0, 1, 2, or many items
- ✂ Each **OrderItem** must belong to exactly one **Order**

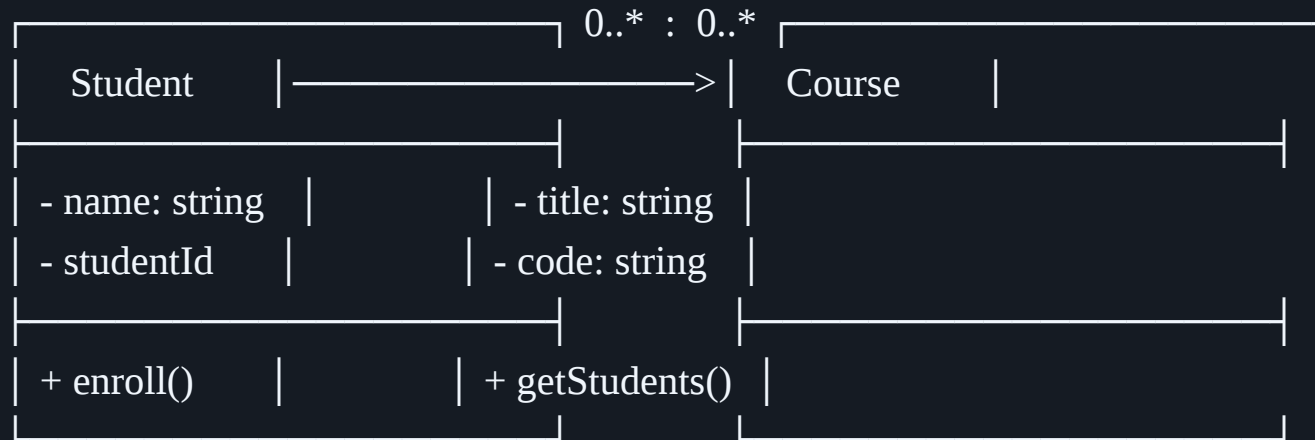
PERSON AND PASSPORT (ONE-TO-ONE)



Read:

- ✂ One **Person** has exactly one **Passport** (1:1)
- ✂ One **Passport** belongs to exactly one **Person**

STUDENT AND COURSE (MANY-TO-MANY)



Read:

- ✂ One Student can enroll in many Courses
- ✂ One Course can have many Students

Relationships - Association

A RELATIONSHIP BETWEEN INDEPENDENT OBJECTS



Key Differences from HAS-A:

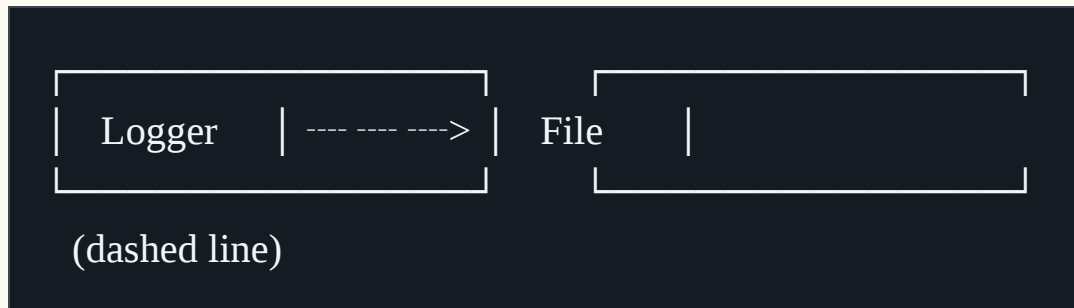
- ✘ No containment (not a part-of relationship)
- ✘ Objects are independent
- ✘ Can be created/destroyed independently
- ✘ Represents "uses" or "knows about"

Example:

- ✘ A Student is NOT "part of" a Course
- ✘ A Student enrolls in a Course
- ✘ Both exist independently

RELATIONSHIPS - DEPENDENCY 1/2

Weakest relationship: temporary or method-level usage



Characteristics:

- ✘ Dashed line (not solid)
- ✘ No field storage
- ✘ Object used temporarily (parameter, local variable)
- ✘ Weak coupling

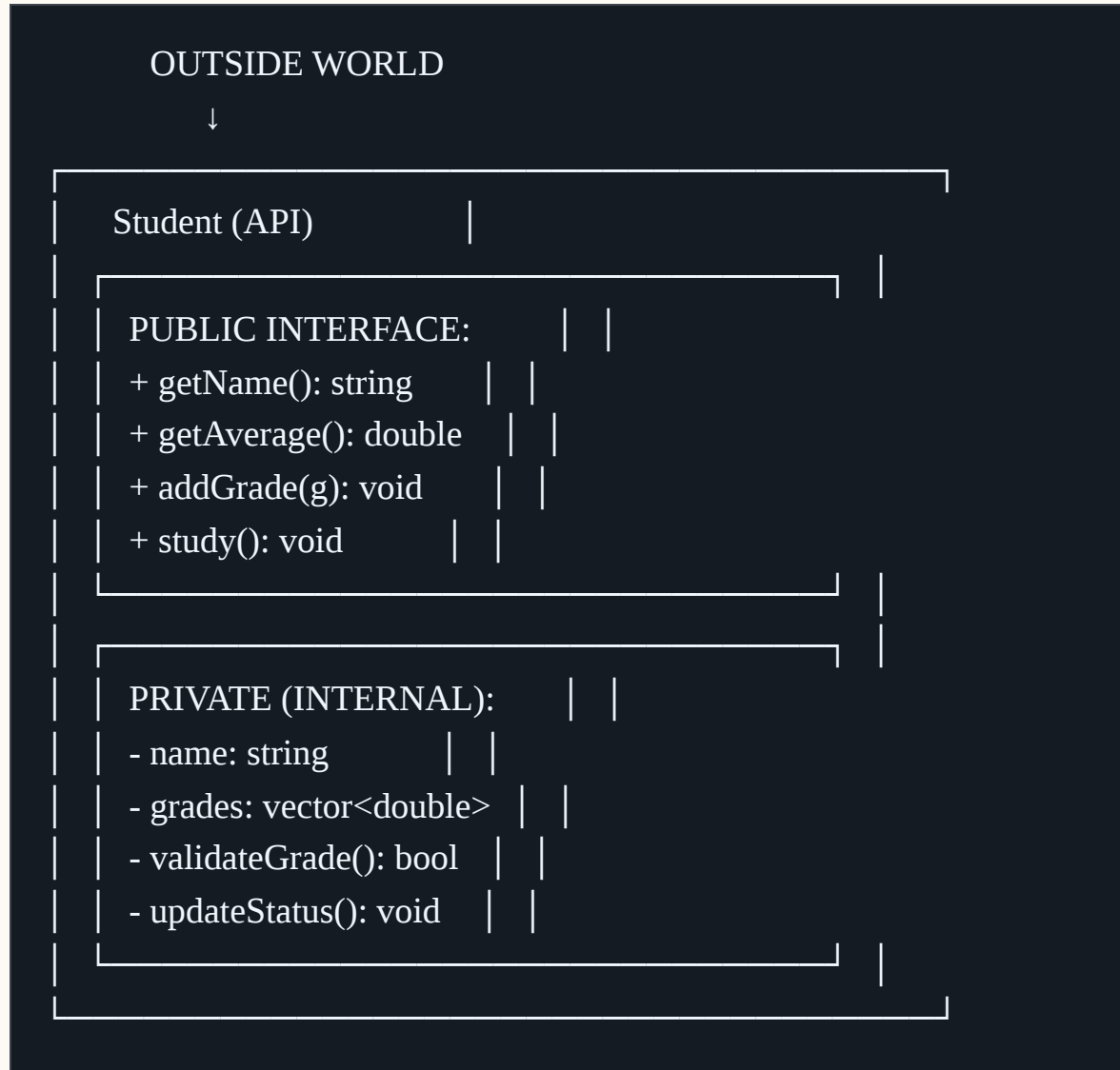
RELATIONSHIPS - DEPENDENCY 2/2

```
class Logger
{
    public: void write(File& file)
    { // Dependency: File passed as parameter
        file.open();
        file.write("log message");
        file.close();
    }
};

// Logger doesn't own File, just uses it temporarily
```



Encapsulation - On the Diagram



Principle:

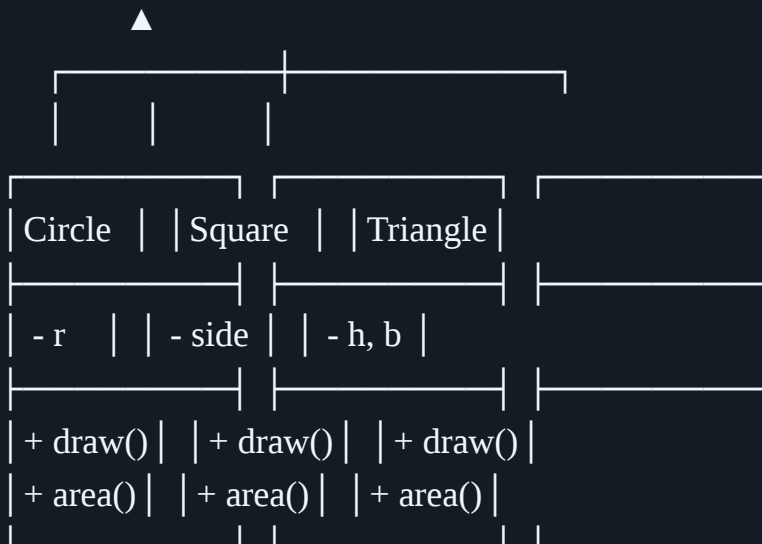
- ✖ Outside world sees: **+** methods only (API)
- ✖ Outside world CANNOT see: **-** fields (hidden)
- ✖ Outside world CANNOT see: **-** methods (internal helpers)
- ✖ **Safety:** no direct field modification
- ✖ **Validation:** all changes go through public methods

Abstraction - Abstract Classes

```

<<abstract>>
Shape
# color: string
+ draw(): void (virtual)
+ getArea(): double

```



Abstract Class Rules:

- ✖ `<<interface>>` = abstract class
- ✖ Pure virtual methods = must implement in subclass
- ✖ Cannot create: `new Shape()` ❌
- ✖ But: `Shape* s = new Circle()` ✅
(polymorphism)

ABSTRACTION - ABSTRACT CLASSES

```
class Shape
{ // abstract
    public: virtual void draw() = 0; // pure virtual
    public: virtual double getArea() = 0;
};

class Circle : public Shape
{
    public: void draw() override
    {
        /* circle drawing */
    }
    public: double getArea() override
    {
        return 3.14 * r * r;
    }
};
```

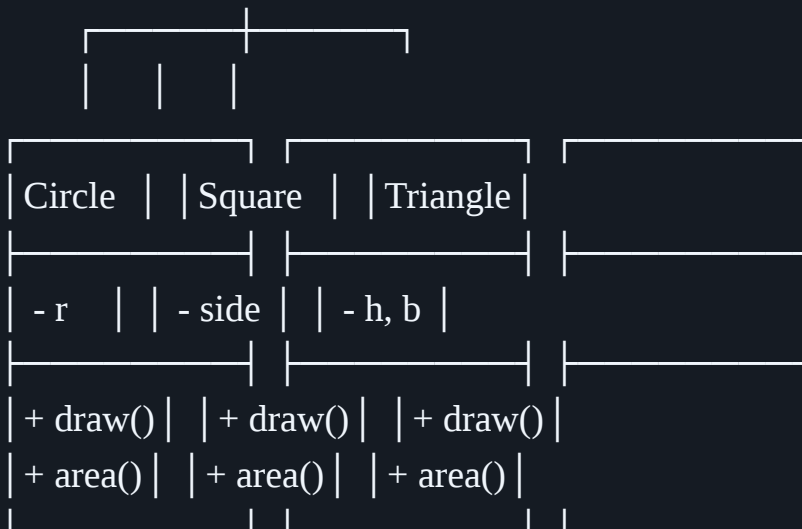
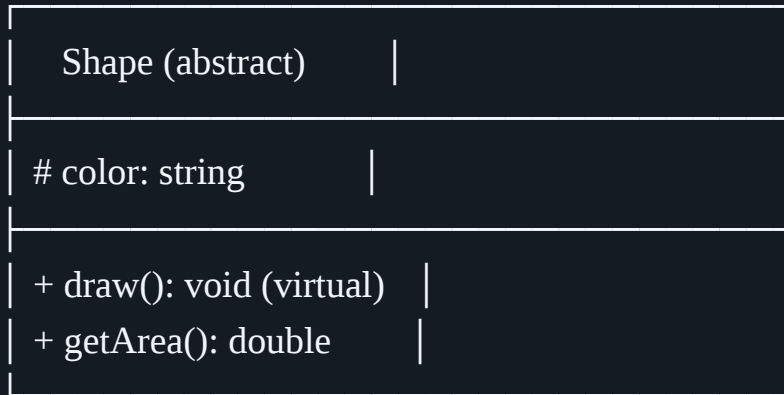


Polymorphism - On the Diagram

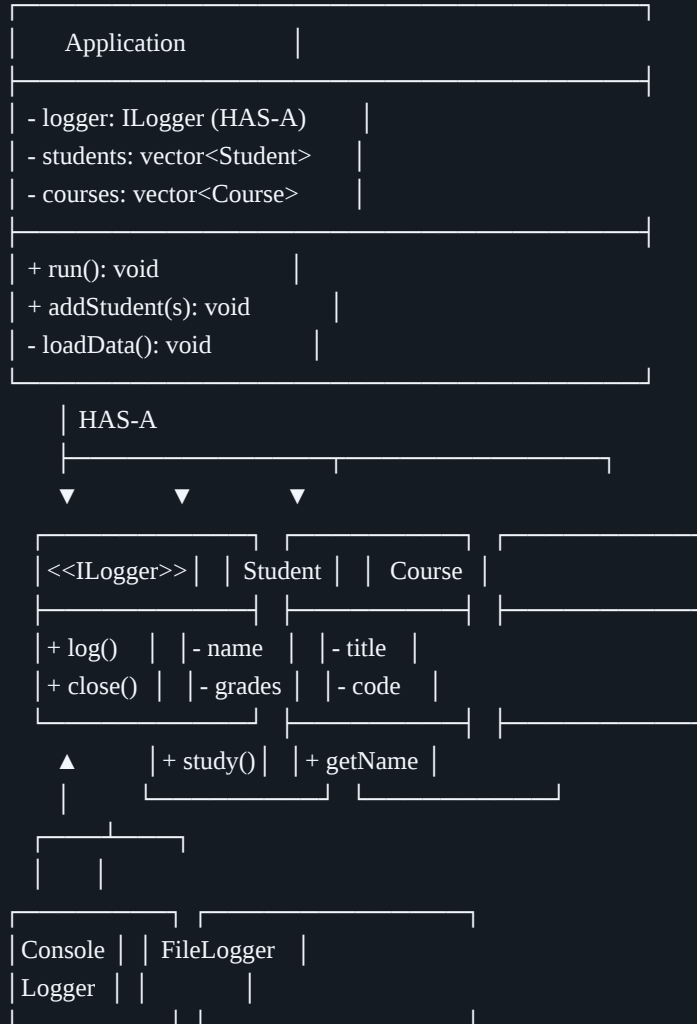
Polymorphism = "many forms"

- ✖ **Each subclass:** same interface, different implementation
- ✖ **Runtime dispatch:** calls correct version
- ✖ **Example:** `Shape* s = new Circle();` `s->draw();` // `>`
`Circle::draw()!`

```
Shape* s = new Circle();
s->draw(); // > calls Circle::draw(), not Shape::draw()!
```



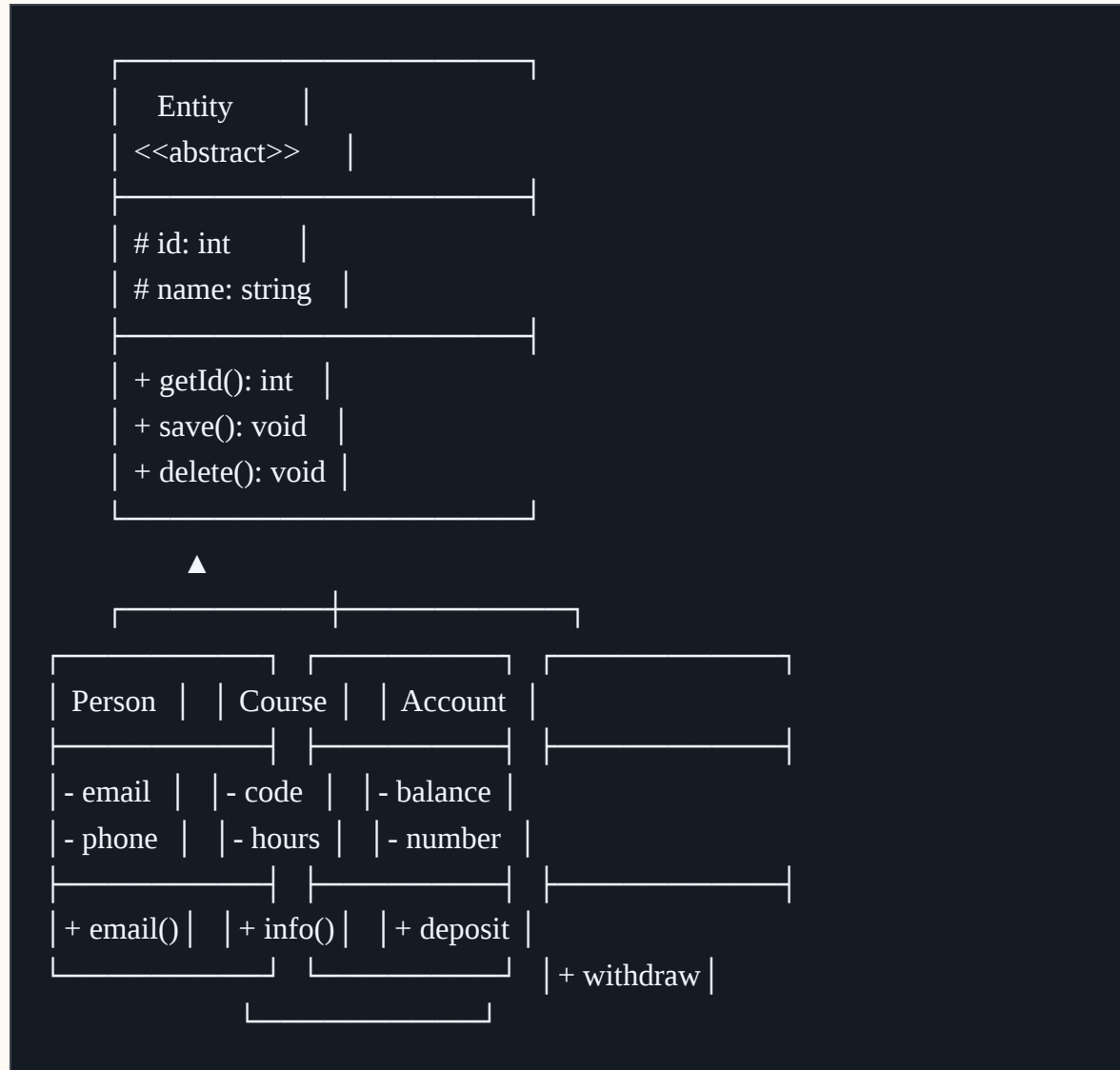
Complete System Example



Relationships:

- ✖ App HAS-A Logger (dependency)
- ✖ App HAS-A many Students (composition/aggregation)
- ✖ App HAS-A many Courses (composition/aggregation)
- ✖ Student enrolls in Courses (association, many-to-many)
- ✖ ILogger is abstract (interface)
- ✖ ConsoleLogger and FileLogger implement ILogger (inheritance)

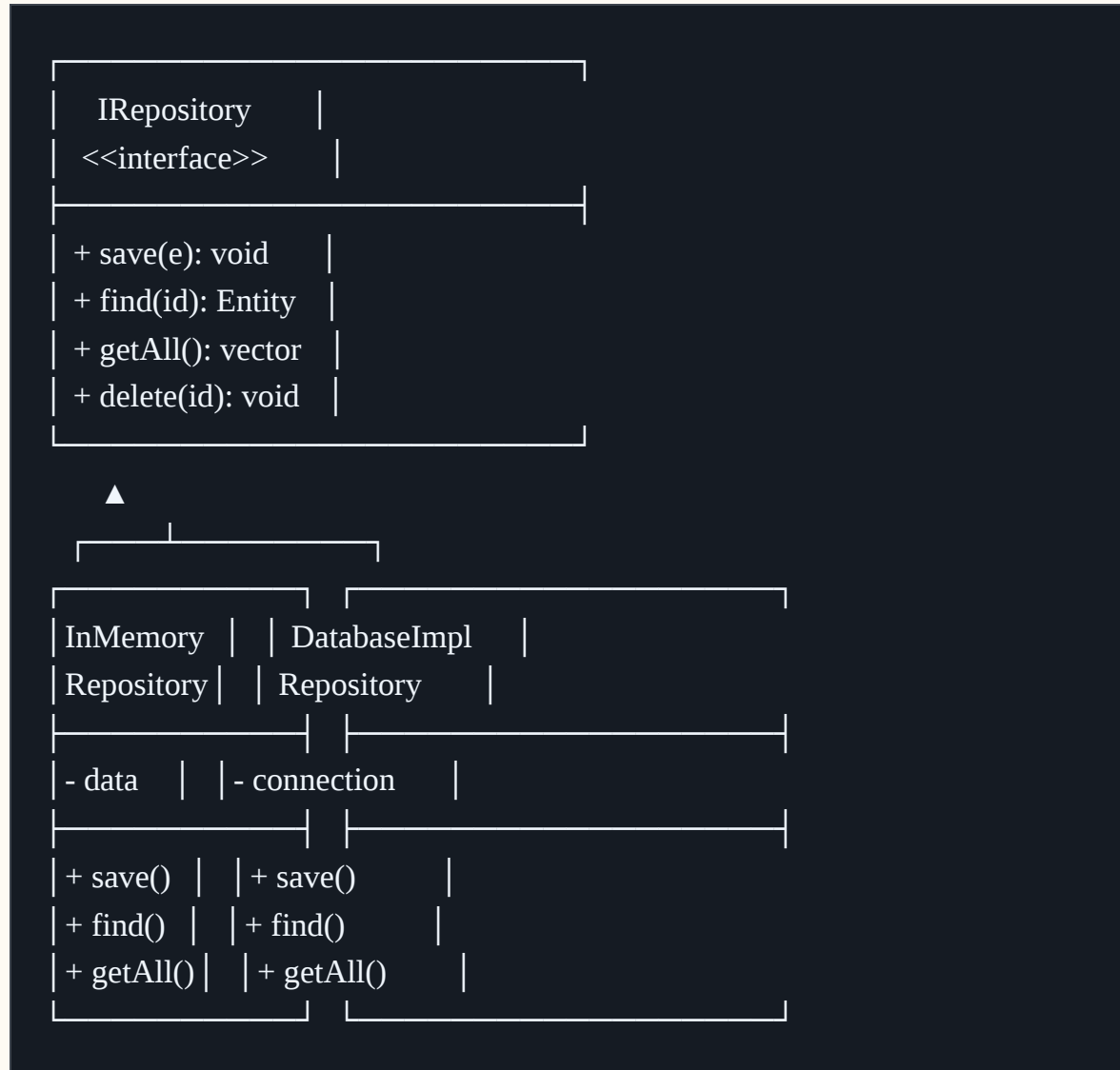
Database Architecture Example



Inheritance Hierarchy:

- ✖ All inherit from Entity (IS-A)
- ✖ All have `id`, `name`
- ✖ All can `save()`, `delete()`
- ✖ Each adds own specific fields

Repository Pattern



Abstract interface:

- ✖ Can swap implementations
- ✖ In-memory for testing
- ✖ Database for production

SUMMARY - KEY CONCEPTS 1/2

- ✂ **Access Modifiers** (+ , - , #)
 - ✂ Control visibility
 - ✂ Enforce encapsulation
- ✂ **Encapsulation**
 - ✂ Private fields
 - ✂ Public methods
 - ✂ Protection through validation

- ✂ **Inheritance** (IS-A)
 - ✂ Reuse code
 - ✂ Create hierarchies
 - ✂ Empty triangle in UML
- ✂ **Composition** (HAS-A (Strong))
 - ✂ Objects contain objects
 - ✂ Part cannot exist without whole
 - ✂ Lifetime dependency
 - ✂ Broken lines in UML
 - ✂ Filled diamond in UML

Summary - Key Concepts 2/2

✂ Aggregation (HAS-A Weak)

- ✂ Empty diamond in UML
- ✂ Objects are independent
- ✂ Can exist separately

✂ Dependency

- ✂ Dashed line (-----)
- ✂ Temporary usage
- ✂ A doesn't own B, just uses it temporarily
- ✂ Weakest coupling

✂ Multiplicity / Cardinality

- ✂ 1, *, 0..1, 1..*

- ✂ Defines "how many"

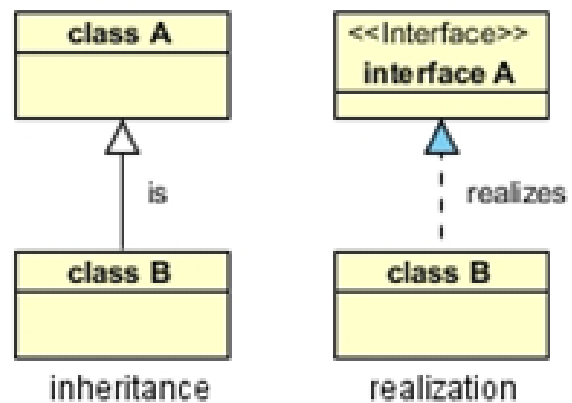
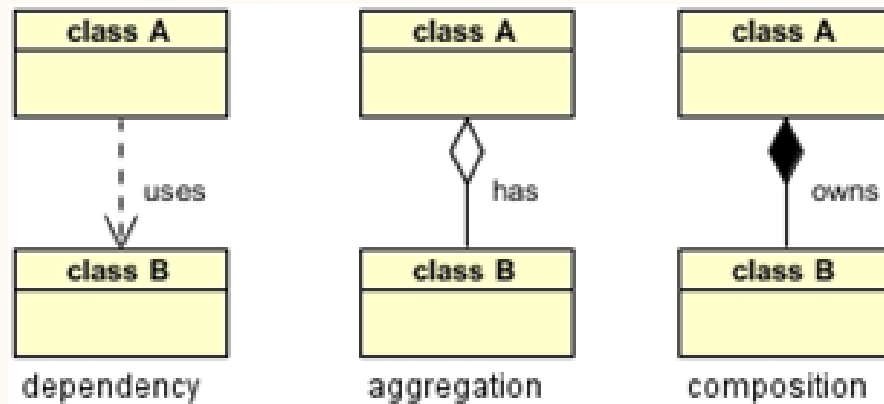
- ✂ Critical for design

✂ Abstraction

- ✂ Abstract classes
- ✂ Interfaces
- ✂ Pure virtual methods

✂ Polymorphism

- ✂ Virtual methods
- ✂ Override behavior ; Runtime dispatch³⁷



THANK

You